1. Tables -
    - **Customers**
        - Column & Types-
            - Customer_id INT
            - First_name VARCHAR
            - Last_name VARCHAR
            - Email VARCHAR

    Name is composite since it's broken up into First_name and Last_name.

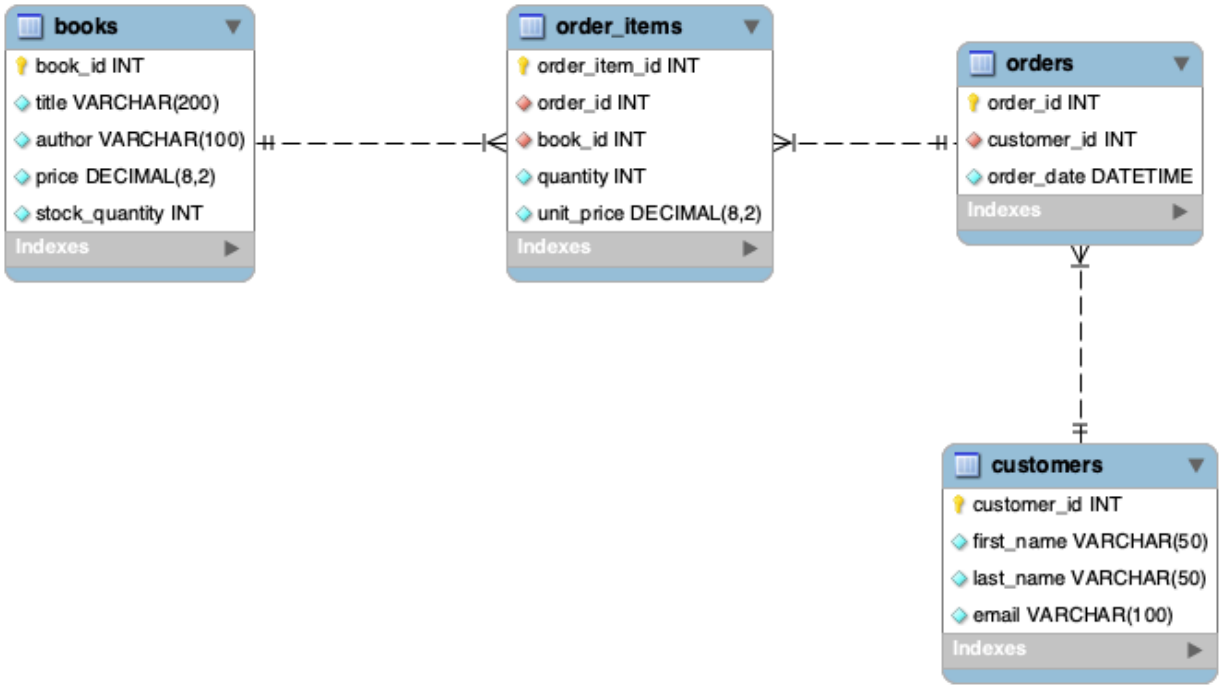
    - **Books**
        - Colun & Types-
            - Book_id INT
            - Title VARCHAR
            - Author VARCHAR
            - Price DECIMAL
            - Stock_quantity INT
    - **Orders**
        - Column & Types-
            - Order_id INT
            - Customer_id INT
            - Order_date DATETIME
    - **Orders_items** (Many to Many connector for Books and Orders)
        - Column & Types-
            - Order_item_id INT
            - Order_id INT
            - Book_id INT
            - Quantity INT
            - Unit_price DECIMAL

    Derived attribute is: line_total (quantity * unit_price)

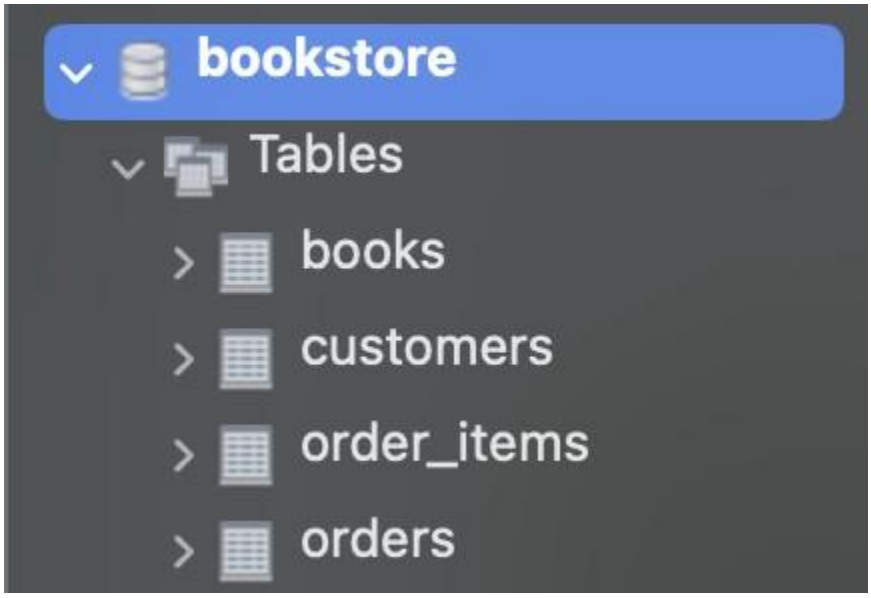
2. Primary keys
   Customers - customer_id - auto-increment ID, unique, email and name could repeat.
   Books - book_id - same reasoning
   Orders - order_id - each order needs its own ID separate from customer or date
   Order items - order_item_id - simple, composite key of (order_id, book_id also works)

3. Relationship & Carddinality
   Customer -> Oders: 1:M, one customer can place many orders but each order belongs to only one customer.
   Orders <-> Books: M:M, an order can contain many books and the same book can appear in many different orders.

4.



5. Redundant - Customer, Email, Book, and price repeat on every line
   Customer “Suda” and her email appear twice, the book “Databases 101” / 450 appear twice.
   Insertion / deletion - can't record a new book or a new customer until they appear on an order, deleting the one order

6. 

7. Foreign Key is a column (or set of columns) in one table that match a value of the primary key in another table, its relational database referencing.
   In this design **Order_items.book_id** REFERENCES **books.book_id** and that would reject Insertion / update on **Order_items.book_id** that doesn't match with **books.book_id** preventing order that points to nothing.

8. Relational DB vs in-memory array
   In memory array only exists when your program is running, when it stops, the things in it is gone. A database saves data to disk so its still there the next time you run the program, or even ifthe program crashes halfway through.

   The data survives.